



SMART CONTRACT AUDIT REPORT

for

Yild Finance



Prepared By: Xiaomi Huang

PeckShield
October 1, 2025

Document Properties

Client	Yild Finance
Title	Smart Contract Audit Report
Target	Yild Finance
Version	1.0
Author	Xuxian Jiang
Auditors	Matthew Jiang, Xuxian Jiang
Reviewed by	Xiaomi Huang
Approved by	Xuxian Jiang
Classification	Public

Version Info

Version	Date	Author(s)	Description
1.0	October 1, 2025	Xuxian Jiang	Final Release
1.0-rc	October 1, 2025	Xuxian Jiang	Release Candidate #1

Contact

For more information about this document and its contents, please contact PeckShield Inc.

Name	Xiaomi Huang
Phone	+86 183 5897 7782
Email	contact@peckshield.com

Contents

1	Introduction	4
1.1	About Yild	4
1.2	About PeckShield	5
1.3	Methodology	5
1.4	Disclaimer	7
2	Findings	9
2.1	Summary	9
2.2	Key Findings	10
3	Detailed Results	11
3.1	Stale State Cleanup After Rebalancing User Positions	11
3.2	Missing Protocol Fee Collection in PositionDecreaser	12
3.3	Improved Protocol Fee Calculation in PositionCollector	14
3.4	Accommodation of Non-ERC20-Compliant Tokens	15
3.5	Improved New Position Creation Logic After Rebalance	17
4	Conclusion	19
	References	20

1 | Introduction

Given the opportunity to review the design document and related smart contract source code of the Yild protocol, we outline in the report our systematic approach to evaluate potential security issues in the smart contract implementation, expose possible semantic inconsistencies between smart contract code and design document, and provide additional suggestions or recommendations for improvement. Our results show that the given version of smart contracts can be further improved due to the presence of several issues related to either security or performance. This document outlines our audit results.

1.1 About Yild

Yild Finance is a Uniswap V3 position management protocol designed to optimize liquidity provision strategies through automation. The protocol prioritizes capital protection by integrating single-click liquidity management and swaps. This audit covers the new rebalancer contract that allows to re-adjust when the provided liquidity is out-of-range. The basic information of the audited protocol is as follows:

Table 1.1: Basic Information of The Yild Finance

Item	Description
Name	Yild Finance
Type	EVM Smart Contract
Platform	Solidity
Audit Method	Whitebox
Latest Audit Report	October 1, 2025

In the following, we show the Git repository of reviewed files and the commit hash value used in this audit:

- <https://github.com/YildFinance/cYild.git> (da6e7e8)

And here is the commit ID after fixes for the issues found in the audit have been checked in:

- <https://github.com/YildFinance/cYild.git> (0171893)

1.2 About PeckShield

PeckShield Inc. [8] is a leading blockchain security company with the goal of elevating the security, privacy, and usability of current blockchain ecosystems by offering top-notch, industry-leading services and products (including the service of smart contract auditing). We are reachable at Telegram (<https://t.me/peckshield>), Twitter (<http://twitter.com/peckshield>), or Email (contact@peckshield.com).

Table 1.2: Vulnerability Severity Classification

Impact	High	Critical	High	Medium
	Medium	High	Medium	Low
	Low	Medium	Low	Low
		High	Medium	Low
		Likelihood		

1.3 Methodology

To standardize the evaluation, we define the following terminology based on OWASP Risk Rating Methodology [7]:

- Likelihood represents how likely a particular vulnerability is to be uncovered and exploited in the wild;
- Impact measures the technical loss and business damage of a successful attack;
- Severity demonstrates the overall criticality of the risk.

Likelihood and impact are categorized into three ratings: *H*, *M* and *L*, i.e., *high*, *medium* and *low* respectively. Severity is determined by likelihood and impact and can be classified into four categories accordingly, i.e., *Critical*, *High*, *Medium*, *Low* shown in Table 1.2.

To evaluate the risk, we go through a list of check items and each would be labeled with a severity category. For one check item, if our tool or analysis does not identify any issue, the contract is considered safe regarding the check item. For any discovered issue, we might further

Table 1.3: The Full List of Check Items

Category	Check Item
Basic Coding Bugs	Constructor Mismatch
	Ownership Takeover
	Redundant Fallback Function
	Overflows & Underflows
	Reentrancy
	Money-Giving Bug
	Blackhole
	Unauthorized Self-Destruct
	Revert DoS
	Unchecked External Call
	Gasless Send
	Send Instead Of Transfer
	Costly Loop
	(Unsafe) Use Of Untrusted Libraries
	(Unsafe) Use Of Predictable Variables
	Transaction Ordering Dependence
	Deprecated Uses
Semantic Consistency Checks	Semantic Consistency Checks
Advanced DeFi Scrutiny	Business Logics Review
	Functionality Checks
	Authentication Management
	Access Control & Authorization
	Oracle Security
	Digital Asset Escrow
	Kill-Switch Mechanism
	Operation Trails & Event Generation
	ERC20 Idiosyncrasies Handling
	Frontend-Contract Integration
	Deployment Consistency
Additional Recommendations	Holistic Risk Management
	Avoiding Use of Variadic Byte Array
	Using Fixed Compiler Version
	Making Visibility Level Explicit
	Making Type Inference Explicit
	Adhering To Function Declaration Strictly
	Following Other Best Practices

deploy contracts on our private testnet and run tests to confirm the findings. If necessary, we would additionally build a PoC to demonstrate the possibility of exploitation. The concrete list of check items is shown in Table 1.3.

In particular, we perform the audit according to the following procedure:

- Basic Coding Bugs: We first statically analyze given smart contracts with our proprietary static code analyzer for known coding bugs, and then manually verify (reject or confirm) all the issues found by our tool.
- Semantic Consistency Checks: We then manually check the logic of implemented smart contracts and compare with the description in the white paper.
- Advanced DeFi Scrutiny: We further review business logics, examine system operations, and place DeFi-related aspects under scrutiny to uncover possible pitfalls and/or bugs.
- Additional Recommendations: We also provide additional suggestions regarding the coding and development of smart contracts from the perspective of proven programming practices.

To better describe each issue we identified, we categorize the findings with Common Weakness Enumeration (CWE-699) [6], which is a community-developed list of software weakness types to better delineate and organize weaknesses around concepts frequently encountered in software development. Though some categories used in CWE-699 may not be relevant in smart contracts, we use the CWE categories in Table 1.4 to classify our findings.

1.4 Disclaimer

Note that this security audit is not designed to replace functional tests required before any software release, and does not give any warranties on finding all possible security issues of the given smart contract(s) or blockchain software, i.e., the evaluation result does not guarantee the nonexistence of any further findings of security issues. As one audit-based assessment cannot be considered comprehensive, we always recommend proceeding with several independent audits and a public bug bounty program to ensure the security of smart contract(s). Last but not least, this security audit should not be used as investment advice.

Table 1.4: Common Weakness Enumeration (CWE) Classifications Used in This Audit

Category	Summary
Configuration	Weaknesses in this category are typically introduced during the configuration of the software.
Data Processing Issues	Weaknesses in this category are typically found in functionality that processes data.
Numeric Errors	Weaknesses in this category are related to improper calculation or conversion of numbers.
Security Features	Weaknesses in this category are concerned with topics like authentication, access control, confidentiality, cryptography, and privilege management. (Software security is not security software.)
Time and State	Weaknesses in this category are related to the improper management of time and state in an environment that supports simultaneous or near-simultaneous computation by multiple systems, processes, or threads.
Error Conditions, Return Values, Status Codes	Weaknesses in this category include weaknesses that occur if a function does not generate the correct return/status code, or if the application does not handle all possible return/status codes that could be generated by a function.
Resource Management	Weaknesses in this category are related to improper management of system resources.
Behavioral Issues	Weaknesses in this category are related to unexpected behaviors from code that an application uses.
Business Logics	Weaknesses in this category identify some of the underlying problems that commonly allow attackers to manipulate the business logic of an application. Errors in business logic can be devastating to an entire application.
Initialization and Cleanup	Weaknesses in this category occur in behaviors that are used for initialization and breakdown.
Arguments and Parameters	Weaknesses in this category are related to improper use of arguments or parameters within function calls.
Expression Issues	Weaknesses in this category are related to incorrectly written expressions within code.
Coding Practices	Weaknesses in this category are related to coding practices that are deemed unsafe and increase the chances that an exploitable vulnerability will be present in the application. They may not directly introduce a vulnerability, but indicate the product has not been carefully developed or maintained.

2 | Findings

2.1 Summary

Here is a summary of our findings after analyzing the design and implementation of the `v11a` protocol. During the first phase of our audit, we study the smart contract source code and run our in-house static code analyzer through the codebase. The purpose here is to statically identify known coding bugs, and then manually verify (reject or confirm) issues reported by our tool. We further manually review business logics, examine system operations, and place DeFi-related aspects under scrutiny to uncover possible pitfalls and/or bugs.

Severity	# of Findings	
Critical	0	
High	0	
Medium	1	
Low	4	
Informational	0	
Total	5	

We have so far identified a list of potential issues: some of them involve subtle corner cases that might not be previously thought of, while others refer to unusual interactions among multiple contracts. For each uncovered issue, we have therefore developed test cases for reasoning, reproduction, and/or verification. After further analysis and internal discussion, we determined a few issues of varying severities that need to be brought up and paid more attention to, which are categorized in the above table. More information can be found in the next subsection, and the detailed discussions of each of them are in [Section 3](#).

2.2 Key Findings

Overall, these smart contracts are well-designed and engineered, though the implementation can be improved by resolving the identified issues (shown in Table 2.1), including 1 medium-severity vulnerability and 4 low-severity vulnerabilities.

Table 2.1: Key Yild Finance Audit Findings

ID	Severity	Title	Category	Status
PVE-001	Low	Stale State Cleanup After Rebalancing User Positions	Coding Practices	Resolved
PVE-002	Medium	Missing Protocol Fee Collection in PositionDecreaser	Business Logic	Resolved
PVE-003	Low	Improved Protocol Fee Calculation in PositionCollector	Business Logic	Resolved
PVE-004	Low	Accommodation of Non-ERC20-Compliant Tokens	Coding Practices	Resolved
PVE-005	Low	Improved New Position Creation Logic After Rebalance	Business Logic	Resolved

Besides recommending specific countermeasures to mitigate these issues, we also emphasize that it is always important to develop necessary risk-control mechanisms and make contingency plans, which may need to be exercised before the mainnet deployment. The risk-control mechanisms need to kick in at the very moment when the contracts are being deployed in mainnet. Please refer to Section 3 for details.

3 | Detailed Results

3.1 Stale State Cleanup After Rebalancing User Positions

- ID: PVE-001
- Severity: Low
- Likelihood: Low
- Impact: Low
- Target: PositionRebalancer
- Category: Coding Practices [4]
- CWE subcategory: CWE-1041 [1]

Description

The Yild protocol has a clean approach in managing different position-managing functions. In the process of examining new position-rebalancing logic, we notice an issue that may leave stale state unattended.

In the following, we show below the code snippet of the related routine, i.e., `PositionRebalancer()`. We notice the old position `_tokenId` is no longer used while it is still present in the `owners` and `transferApproved` mappings. Though the unattended storage states do not introduce any negative security impact, it is still better to release the occupied storage.

```
73 // Rebalance position funds to 50/50 around current tick
74 (
75     positionRebalanceData.amount0D,
76     positionRebalanceData.amount1D
77 ) = _rebalancePosition(
78     posInfo,
79     positionRebalanceData.newTickLower,
80     positionRebalanceData.newTickUpper,
81     positionRebalanceData.capital0 +
82         positionRebalanceData.feesToken0 -
83         positionRebalanceData.protocolFee0,
84     positionRebalanceData.capital1 +
85         positionRebalanceData.feesToken1 -
86         positionRebalanceData.protocolFee1,
87     _pSwapData0,
88     _pSwapData1,
```

```

89         _minAmountOut0,
90         _minAmountOut1
91     );
92
93     // Create new position and handle cleanup
94     uint256 _newPositionTokenId = _createNewPosition(
95         posInfo,
96         positionRebalanceData.newTickLower,
97         positionRebalanceData.newTickUpper,
98         positionRebalanceData.amount0D,
99         positionRebalanceData.amount1D,
100         _token0MaxSlippage,
101         _token1MaxSlippage,
102         positionRebalanceData.owner
103     );

```

Listing 3.1: PositionRebalancer::PositionRebalancer()

Recommendation Improve the above PositionRebalancer() routine to properly release unused storage states.

Status This issue has been resolved by the following commit: 660a014.

3.2 Missing Protocol Fee Collection in PositionDecreaser

- ID: PVE-002
- Severity: Medium
- Likelihood: Medium
- Impact: Low
- Target: PositionDecreaser
- Category: Business Logic [5]
- CWE subcategory: CWE-841 [3]

Description

By design, Yild may charge protocol fee based on the accumulated swap fee from managed liquidity positions. In the process of examining various operations upon liquidity positions, we notice the position-decreasing operation does not collect protocol fee from the accumulated swap fee.

To elaborate, we show below the code snippet from the related decreaseLiquidity() routine. While a user's position liquidity is properly reduced and the funds are transferred back to user, the accumulated fee may be entitled for protocol fee collection, and is currently not collected.

```

53     nonfungiblePositionManager.decreaseLiquidity(
54         INonfungiblePositionManager.DecreaseLiquidityParams({
55             tokenId: _tokenId,
56             liquidity: _liquidityAmount,
57             amount0Min: LiquidityMath.applySlippage(
58                 _amount0,

```

```

59         _token0MaxSlippage
60     ),
61     amount1Min: LiquidityMath.applySlippage(
62         _amount1,
63         _token1MaxSlippage
64     ),
65     deadline: block.timestamp
66 })
67 );
68
69 (_amount0, _amount1) = UniswapV3NFTHelper.collect(
70     address(nonfungiblePositionManager),
71     _tokenId
72 );
73
74 _refundToken(
75     posInfo.token0,
76     owners[_tokenId],
77     _amount0,
78     _minAmountOut0,
79     _pSwapData0
80 );
81 _refundToken(
82     posInfo.token1,
83     owners[_tokenId],
84     _amount1,
85     _minAmountOut1,
86     _pSwapData1
87 );
88
89 _sweep(_tokenId, posInfo.token0, posInfo.token1);

```

Listing 3.2: PositionDecreaser::decreaseLiquidity()

Recommendation Revisit the above routine to properly collect protocol fee when a liquidity position is decreased.

Status This issue has been resolved by the following commit: 0d7fa4f.

3.3 Improved Protocol Fee Calculation in PositionCollector

- ID: PVE-003
- Severity: Low
- Likelihood: Low
- Impact: Low
- Target: PositionCollector
- Category: Business Logic [5]
- CWE subcategory: CWE-841 [3]

Description

As mentioned in Section 3.2, `Yild` is designed to charge protocol fee based on the accumulated swap fee from managed positions. In the process of examining the specific collector operation, we notice the protocol fee amount is not accurately calculated.

To elaborate, we show below the implementation from the related `collectFees()` routine. As the name indicates, this routine allows the position owner to collect the swap fee from the provided liquidity. It comes to our attention that the protocol fee is computed based on the `posInfo.tokensOwed0 / 1` (lines 30 – 31), which should be revised as `feesToken0 / 1`.

```

10     function collectFees(
11         uint256 tokenId
12     )
13     external
14     onlyOwner(tokenId)
15     returns (uint256 feesToken0, uint256 feesToken1)
16     {
17         PositionInfo memory posInfo = UniswapV3NFTHelper.getPositionInfo(
18             tokenId,
19             address(nonfungiblePositionManager)
20         );
21
22         (feesToken0, feesToken1) = UniswapV3NFTHelper.collect(
23             address(nonfungiblePositionManager),
24             tokenId
25         );
26
27         (uint256 protocolFee0, uint256 protocolFee1) = _collectProtocolFee(
28             posInfo.token0,
29             posInfo.token1,
30             posInfo.tokensOwed0,
31             posInfo.tokensOwed1
32         );
33
34         // Transfer remaining fees to recipient
35         uint256 remainingToken0 = feesToken0 - protocolFee0;
36         uint256 remainingToken1 = feesToken1 - protocolFee1;
37
38         if (remainingToken0 > 0) {

```

```

39         _transferToken(posInfo.token0, owners[tokenId], remainingToken0);
40     }
41     if (remainingToken1 > 0) {
42         _transferToken(posInfo.token1, owners[tokenId], remainingToken1);
43     }
44
45     _sweep(tokenId, posInfo.token0, posInfo.token1);
46
47     emit FeesCollected(tokenId, feesToken0, feesToken1);
48 }

```

Listing 3.3: PositionCollector::collectFees()

Recommendation Revisit the above routine to properly compute protocol fee when a liquidity position's swap fee is being collected.

Status This issue has been resolved by the following commit: 7882871.

3.4 Accommodation of Non-ERC20-Compliant Tokens

- ID: PVE-004
- Severity: Low
- Likelihood: Low
- Impact: Low
- Target: Multiple Contracts
- Category: Coding Practices [4]
- CWE subcategory: CWE-563 [2]

Description

Though there is a standardized ERC-20 specification, many token contracts may not strictly follow the specification or have additional functionalities beyond the specification. In the following, we examine the `transfer()` routine and related idiosyncrasies from current widely-used token contracts.

In particular, we use the popular token, i.e., ZRX, as our example. We show the related code snippet below. On its entry of `transfer()`, there is a check, i.e., `if (balances[msg.sender] >= _value && balances[_to] + _value >= balances[_to])`. If the check fails, it returns `false`. However, the transaction still proceeds successfully without being reverted. This is not compliant with the ERC20 standard and may cause issues if not handled properly. Specifically, the ERC20 standard specifies the following: “Transfers `_value` amount of tokens to address `_to`, and *MUST* fire the Transfer event. The function *SHOULD* throw if the message caller's account balance does not have enough tokens to spend.”

```

64     function transfer(address _to, uint _value) returns (bool) {
65         //Default assumes totalSupply can't be over max (2^256 - 1).
66         if (balances[msg.sender] >= _value && balances[_to] + _value >= balances[_to]) {
67             balances[msg.sender] -= _value;

```

```

68         balances[_to] += _value;
69         Transfer(msg.sender, _to, _value);
70         return true;
71     } else { return false; }
72 }

74 function transferFrom(address _from, address _to, uint _value) returns (bool) {
75     if (balances[_from] >= _value && allowed[_from][msg.sender] >= _value &&
76         balances[_to] + _value >= balances[_to]) {
77         balances[_to] += _value;
78         balances[_from] -= _value;
79         allowed[_from][msg.sender] -= _value;
80         Transfer(_from, _to, _value);
81         return true;
82     } else { return false; }
}

```

Listing 3.4: ZRX.sol

Because of that, a normal call to `transfer()` is suggested to use the safe version, i.e., `safeTransfer()`. In essence, it is a wrapper around ERC20 operations that may either throw on failure or return false without reverts. Moreover, the safe version also supports tokens that return no value (and instead revert or throw on failure). Note that non-reverting calls are assumed to be successful. Similarly, there is a safe version of `approve()/transferFrom()` as well, i.e., `safeApprove()/safeTransferFrom()`.

In the following, we show an example routine in the `UniswapV3SwapManager` contract. If the USDT token is supported as `_sellToken`, the unsafe version of `ERC20(_sellToken).approve(params.router, _sellAmount)` (line 69) may revert as there is no return value in the USDT token contract's `approve()` implementation (but the `IERC20` interface expects a return value)!

```

44 function swap(
45     address _sellToken,
46     address _buyToken,
47     uint256 _sellAmount,
48     uint256 _minBuyAmount,
49     bytes calldata _data
50 )
51 public
52 swapChecks(_sellToken, _buyToken, _sellAmount, _minBuyAmount)
53 onlyDelegateCall
54 returns (uint256)
55 {
56     UniswapV3SwapParams memory params;
57     (
58         params.router,
59         params.uniswapV3factory,
60         params.recipient,
61         params.sqrtPriceLimitX96
62     ) = abi.decode(_data, (address, address, address, uint160));
63
64     params.sellToken = _sellToken;

```

```

65     params.buyToken = _buyToken;
66     params.sellAmount = _sellAmount;
67     params.minBuyAmount = _minBuyAmount;
68
69     ERC20(_sellToken).approve(params.router, _sellAmount);
70
71     return _executeSwap(params);
72 }

```

Listing 3.5: UniswapV3SwapManager::swap()

Recommendation Accommodate the above-mentioned idiosyncrasy about ERC20-related `approve()/transfer()/transferFrom()`. Note this suggestion is also applicable to another function `ParaswapSwapManager::swap()`.

Status This issue has been resolved by the following commit: [e82d6eb](#).

3.5 Improved New Position Creation Logic After Rebalance

- ID: PVE-005
- Severity: Low
- Likelihood: Low
- Impact: Low
- Target: PositionRebalancer
- Category: Business Logic [5]
- CWE subcategory: CWE-841 [3]

Description

As mentioned earlier, `yield` allows an out-of-range position to be rebalanced with a new price range. While reviewing the logic to handle unused funds after rebalancing, we notice the need to properly manage the token allowance on the `nonfungiblePositionManager` contract especially when not all funds are used up.

To elaborate, we show below the implementation from the related `_createNewPosition()` routine. While the rebalancing operation may ideally use up all funds to create a new position, it is possible that not all funds are used. With that, there is a need to reset the token allowance on `nonfungiblePositionManager`. This is necessary to accommodate unconventional idiosyncrasy about widely-used ERC20 tokens, e.g., `USDT`.

```

112     function _createNewPosition(
113         PositionInfo memory posInfo,
114         int24 _newTickLower,
115         int24 _newTickUpper,
116         uint256 _amount0D,
117         uint256 _amount1D,
118         uint256 _token0MaxSlippage,

```

```

119     uint256 _token1MaxSlippage,
120     address _owner
121 ) internal returns (uint256 _newPositionTokenId) {
122     // Approve tokens for new position
123     _approveToken(posInfo.token0, _amount0D);
124     _approveToken(posInfo.token1, _amount1D);
125
126     // Open the new position
127     (_newPositionTokenId, , , ) = nonfungiblePositionManager.mint(
128         INonfungiblePositionManager.MintParams({
129             token0: posInfo.token0,
130             token1: posInfo.token1,
131             fee: posInfo.fee,
132             tickLower: _newTickLower,
133             tickUpper: _newTickUpper,
134             amount0Desired: _amount0D,
135             amount1Desired: _amount1D,
136             amount0Min: LiquidityMath.applySlippage(
137                 _amount0D,
138                 _token0MaxSlippage
139             ),
140             amount1Min: LiquidityMath.applySlippage(
141                 _amount1D,
142                 _token1MaxSlippage
143             ),
144             recipient: address(this),
145             deadline: block.timestamp + 1 hours
146         })
147     );
148
149     owners[_newPositionTokenId] = _owner;
150
151     // Sweep any remaining dust to the owner
152     _sweep(_newPositionTokenId, posInfo.token0, posInfo.token1);
153 }

```

Listing 3.6: PositionRebalancer::_createNewPosition()

Recommendation Revisit the above routine to properly reset token allowance after the rebalance operation.

Status This issue has been resolved by the following commit: 0171893.

4 | Conclusion

In this audit, we have analyzed the design and implementation of the `yild` protocol, which is a `Uniswap V3` position management protocol designed to optimize liquidity provision strategies through automation. The protocol prioritizes capital protection by integrating single-click liquidity management and swaps. The current code base is well structured and neatly organized. Those identified issues are promptly confirmed and addressed.

Meanwhile, we need to emphasize that `Solidity`-based smart contracts as a whole are still in an early, but exciting stage of development. To improve this report, we greatly appreciate any constructive feedbacks or suggestions, on our methodology, audit findings, or potential gaps in scope/coverage.



References

- [1] MITRE. CWE-1041: Use of Redundant Code. <https://cwe.mitre.org/data/definitions/1041.html>.
- [2] MITRE. CWE-563: Assignment to Variable without Use. <https://cwe.mitre.org/data/definitions/563.html>.
- [3] MITRE. CWE-841: Improper Enforcement of Behavioral Workflow. <https://cwe.mitre.org/data/definitions/841.html>.
- [4] MITRE. CWE CATEGORY: Bad Coding Practices. <https://cwe.mitre.org/data/definitions/1006.html>.
- [5] MITRE. CWE CATEGORY: Business Logic Errors. <https://cwe.mitre.org/data/definitions/840.html>.
- [6] MITRE. CWE VIEW: Development Concepts. <https://cwe.mitre.org/data/definitions/699.html>.
- [7] OWASP. Risk Rating Methodology. https://www.owasp.org/index.php/OWASP_Risk_Rating_Methodology.
- [8] PeckShield. PeckShield Inc. <https://www.peckshield.com>.